

PRACTICAL-02

TASK-01

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using `fork()` system call.
- b) Read the user input as shell command.
- c) Use a system call `exec()` to execute the shell command.
- d) Use a `wait()` system call for waiting a parent process.
- e) Display the PIDs of parent and child processes.

```
root@SRIVALLI:~# nano command_executor.c
root@SRIVALLI:~# gcc command_executor.c -o command_executor
root@SRIVALLI:~# ./command_executor
Enter a Linux command: ls

Parent Process
Parent PID: 557
Child PID : 558

Child Process
Child PID : 558
Parent PID: 557

Executing command: ls
OSSP          command_executor.c.save  nano.443.save
command_executor  fork_process          process_state
command_executor.c  fork_process.c        process_state.c

Child process completed.
root@SRIVALLI:~# |
```

REPORT:

Title: Command Execution using `fork()` and `exec()`

The program was developed to create a parent and child process using fork(). The child process executes the Linux command entered by the user using exec(). The parent process waits for the child using wait(). The PID and PPID of both processes were also displayed.

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
root@SRIVALLI:~# uname
Linux
root@SRIVALLI:~# uname -a
Linux SRIVALLI 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu
Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@SRIVALLI:~# lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:              39 bits physical, 48 bits virtual
  Byte Order:                 Little Endian
CPU(s):                      12
  On-line CPU(s) list:       0-11
Vendor ID:                   GenuineIntel
Model name:                  12th Gen Intel(R) Core(TM) i5-1235U
  CPU family:                 6
  Model:                      154
  Thread(s) per core:        2
  Core(s) per socket:        6
  Socket(s):                  1
  Stepping:                   4
  BogomIPS:                   4991.99
  Flags:                      fpu vme de pse tsc msr pae mce cx8 apic sep mtr
r pge mca cmov pat pse36 clflush mmx fxsr sse s
se2 ss ht syscall nx pdpe1gb rdtscp lm constant
_tsc rep_good nopl xtopology tsc_reliable nonst
op_tsc cpuid tsc_known_freq pni pclmulqdq vmx s
sse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe p
opcnt tsc_deadline_timer aes xsave avx f16c rdr
and hypervisor lahf_lm abm 3dnowprefetch ssbd i
brs ibpb stibp ibrs_enhanced tpr_shadow ept vpi
d ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi
2 erms invpcid rdseed adx smap clflushopt clwb
sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni
vnmi umip waitpkg gfni vaes vpclmulqdq rdpid mo
vdiri movdir64b fsrm md_clear serialize ibt flu
sh_lli arch_capabilities
```

```

oot          578  0.0  0.0  8288 4296 pts/0    R+   05:13   0:00 ps aux
oot@SRIVALLI:~# top
top - 05:14:37 up 6 min,  1 user,  load average: 0.10, 0.04, 0.01
tasks:  24 total,   1 running,  23 sleeping,   0 stopped,   0 zombie
Cpu(s):  0.0 us,   0.0 sy,   0.0 ni, 99.9 id,   0.0 wa,   0.0 hi,   0.0 si,   0.
MiB Mem :  7784.6 total,  6391.1 free,   532.1 used,  1016.1 buff/cache
MiB Swap: 2048.0 total,  2048.0 free,    0.0 used.  7252.5 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+
    1 root        20   0   21864   13132   9788 S   0.0   0.2   0:00.63
    2 root        20   0    3180    2204   2072 S   0.0   0.0   0:00.00
    6 root        20   0    3196    2128   2008 S   0.0   0.0   0:00.00
   52 root        19  -1   34060   12624   11536 S   0.0   0.2   0:00.14
  101 root        20   0   25276    6584   5152 S   0.0   0.1   0:00.23
  117 systemd+    20   0   21344   13248   11124 S   0.0   0.2   0:00.10
  118 systemd+    20   0   91036    7964   6996 S   0.0   0.1   0:00.08
  178 root        20   0    4244    2684   2432 S   0.0   0.0   0:00.01
  179 message+    20   0    9676    5424   4684 S   0.0   0.1   0:00.06
  190 root        20   0   17980    8888   7848 S   0.0   0.1   0:00.11
  195 root        20   0 1756628   13836  11444 S   0.0   0.2   0:00.13
  210 root        20   0    3124    1964   1828 S   0.0   0.0   0:00.00
  211 syslog      20   0   222516   5948   4576 S   0.0   0.1   0:00.08
  226 root        20   0  107036  23304  13852 S   0.0   0.3   0:00.19
  325 root        20   0    6676    4512   3740 S   0.0   0.1   0:00.01
  372 root        20   0   20116   11336   9408 S   0.0   0.1   0:00.09
  373 root        20   0   21172    3496   1784 S   0.0   0.0   0:00.00
  396 root        20   0    6080    5324   3692 S   0.0   0.1   0:00.01
  530 root        20   0    3184    1116    980 S   0.0   0.0   0:00.00
  531 root        20   0    3200    1252   1108 S   0.0   0.0   0:00.01
  532 root        20   0    6076    5360   3672 S   0.0   0.1   0:00.02
  581 root        20   0    9300    5692   3508 R   0.0   0.1   0:00.03
  731 root        20   0  370104  20968  17976 S   0.0   0.3   0:00.02
  736 polkitd     20   0  308172    8060   7164 S   0.0   0.1   0:00.02

```

REPORT:

Title: Hardware Resources and OS Services

In this task, Linux commands like `uname`, `lscpu`, `ps`, and `top` were used to check system information, CPU details, running processes, and resource usage. This helped us understand how the operating system manages CPU, memory, storage, and other hardware resources.